Explainable Double DQN for Ultrafiltration Control in Hemodialysis

Selected Topics: Reinforcement Learning & Explainable AI

Gomana Hossam Eldine
ID 231013248

Yousef Mohamed Hasabo
ID 231011574

Mohanad Mostfa Ali
ID 231012398

Zyad Gbriel
ID 231002617

Project idea

An RL agent learns how fast to remove water during dialysis; Integrated Gradients shows why it chose each action.

1

Simulate

Real patient profiles go into a small simulator. One session (episode) = 16 steps of 15 minutes.

2

Learn

A Double DQN agent tries different UF rates (how fast water is removed) and learns from the reward.

3

Explain

Integrated Gradients shows which inputs pushed each choice.

Problem statement: an RL problem

Medical context: dialysis removes extra water from the blood. Removing it too fast makes blood pressure fall.

16 decisions in a row

The agent picks a UF rate every 15 minutes, 16 times per session.

Delayed reward

A fast rate now can drop blood pressure a few steps later.

A trade-off

Remove the water, but keep blood pressure safe.

Why not a fixed rule? It uses one rate all session and does not react when blood pressure falls.

Why look inside? The policy (how the agent picks actions) is a neural network, and we cannot see inside it. We must see which inputs drive each choice.

Literature review

Paper	What it gives our RL work
Yang et al. 2022 [1]	RL (Duel-DDQN) for a dialysis decision: dry weight (target body weight)
van Hasselt et al. 2016 [2]	Double DQN: our learning algorithm
Sundararajan et al. 2017 [3]	Integrated Gradients: our explanation method
Li et al. 2025 [4]	Explainable model that warns of blood-pressure problems
Wang et al. 2025 [5]	Predicts low blood pressure, but no control
Kanbay et al. 2020 [6]	Rules for our simulator and the 13 mL/kg/h limit
Cheungpasitporn et al. 2026 [9]	Review: AI and RL in kidney care
Tang et al. 2026 [10]	Review of RL for treatment: clear S, A, R; need for explanations

The gap: papers [4, 5] only predict risk. No paper explains a learned controller during the session. We do that with DDQN + IG.

Proposed approach: environment and data

UCI CKD data [7, 8]

400 real patient profiles

→

Simulator

4 simple equations

→

RL agent

learns by trial and error

320 training patients

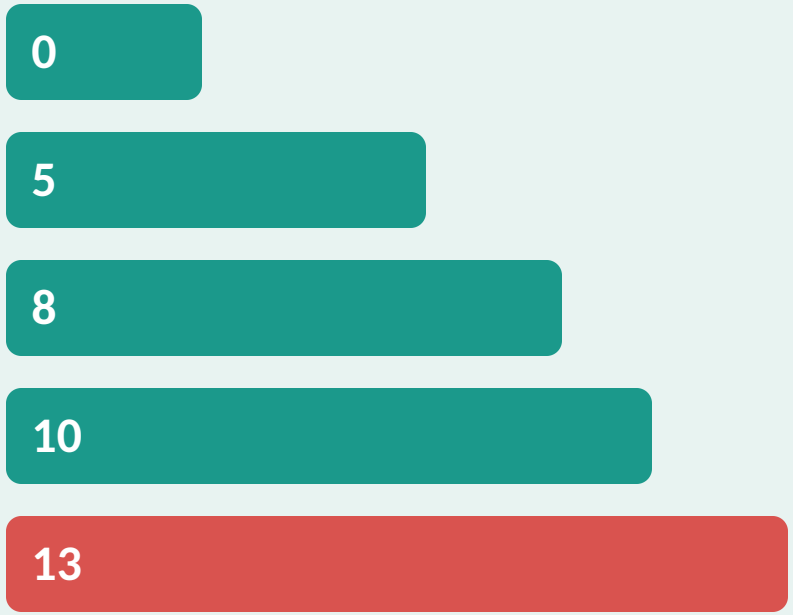
80 test patients (never seen in training)

- From the data: age, diastolic (bottom-number) blood pressure, urine albumin. We do not use the CKD label.
- Why this data: real, open (CC BY 4.0) and widely used; no public minute-by-minute dialysis data exists.

MDP: state S and action A

State feature	Plain meaning
overload	water still to remove (kg)
removed	water removed so far (kg)
sbp	systolic blood pressure (top number)
dbp	diastolic blood pressure (bottom number)
hr	heart rate
time	how far into the session (0 to 1)
age	patient age
albumin	blood protein that helps water refill

Action A: UF rate (mL/kg/h)



0 = stop. 13 = highest safe rate [6].

The agent sees the state, picks an action, and gets a reward. Then it sees the next state.

MDP: transition S → S′ and reward R

(1) $dv = \min(u \cdot W \cdot \Delta t \,/\, 1000, \text{overload});\ \text{overload}' = \text{overload} - dv$

Water removed this step = UF rate × weight × time.

(2) $SBP' = SBP - a\cdot(1 + \text{age_f})\cdot(u - \text{refill}) + b\cdot(SBPo - SBP) + \text{noise}$

Removing water faster than the body refills lowers blood pressure.

(3) $DBP' = DBP + 0.5\cdot(SBP' - SBP) + \text{noise}$

The bottom pressure follows the top pressure.

(4) $HR' = HRo + c\cdot(SBPo - SBP') + \text{noise}$

The heart beats faster when pressure falls.

Reward $R = dv - 10\cdot[SBP' < 90] - 2\cdot\text{overload}$ (last step only)

+ for water removed · −10 for each low-pressure step · −2 per kg of water left at the end

From Q-learning to Double DQN

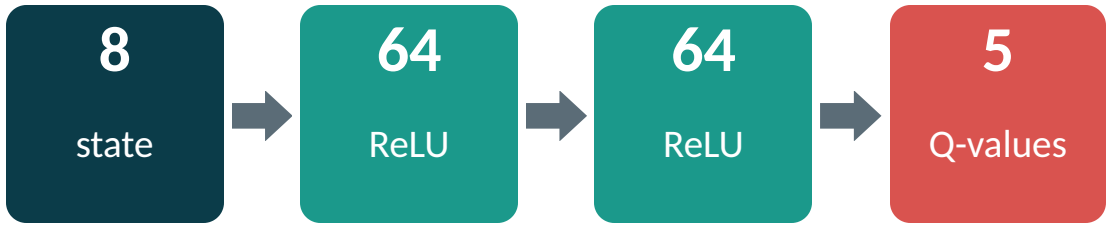
1	**Q(s, a)**	Expected total reward if we take action a in state s, then act well.
2	**Q-learning**	$Q(s, a) \leftarrow r + \gamma \cdot \max Q(s', a')$ (Bellman update)
3	**DQN**	A table cannot store continuous states, so a neural network learns Q.
4	**Problem**	The same network picks and scores the next action, so Q is too high (overestimation).
5	**Double DQN**	Online network picks the action; target network (an older copy) scores it.

DQN target: $y = r + \gamma \cdot \max_a' Q_target(s', a')$

Double DQN: $y = r + \gamma \cdot Q_target(\ s', argmax_a' Q_online(s', a')\ )$

γ (gamma) = how much future reward counts. Picking and scoring with two networks stops the too-high values [2].

Our DDQN agent



In: the 8 state numbers. Out: one Q-value for each of the 5 UF rates. The agent picks the rate with the biggest Q-value.

Part	Our value	Why it is there
Replay buffer	50,000 steps, batch 64	Stores old steps; random batches mix many sessions
ε-greedy	1.0 → 0.05 over first 60% of 1500 episodes	With chance ε, try a random action (explore)
Target network	copied every 200 learning steps	A stable goal to learn toward
Discount γ	0.99	Future reward counts almost as much as now
Optimizer	Adam, lr 1e-3, Huber loss, grad clip 10	Steady updates; big errors do not cause huge jumps

Training loop

```
for seed in 0..4:
  for episode in 1..1500:
    s = reset()              # random training patient
    ε = decay(episode)       # 1.0 → 0.05
    repeat 16 steps:
      a = random if rand < ε else argmax Q(s)
      s', r = step(a)
      buffer.store(s, a, r, s')
      batch = buffer.sample(64)
      update Q_online on DDQN target
      every 200 updates: Q_target ← Q_online
      s = s'
```

5 seeds (0–4)

1500 episodes per seed

16 steps per episode

24,000 steps per seed

Proposed solution: baselines we compare with

Rule-based

No learning. One fixed UF rate per patient to remove the water in 4 h, max 13 mL/kg/h.

What if we do not learn?

Tabular Q-learning

Classic RL with a table. State cut into ranges (bins): 5 water × 5 BP × 16 time steps. 3000 episodes.

Do we need a network?

Standard DQN

Same network and settings as ours, but without the Double fix.

What does the Double fix add?

Double DQN (ours)

Online network picks, target network scores.

Our method.

Proposed solution: the pipeline



If the check fails, main.py stops before IG.

- Why check first? Explaining a model that has not learned well tells us nothing.
- The approval is tied to the exact model file (SHA-256 hash = a fingerprint of the file).
- All seeds are fixed, so the results can be repeated.

```
$ python main.py
```

Results: test patients

Controller	Mean reward ↑	Sessions with low BP ↓	Low-BP steps per session ↓	Water left (kg) ↓	Rate jump ↓
DDQN (ours)	**1.022 ± 0.251**	**1.0% ± 1.5**	**0.010 ± 0.015**	**0.545 ± 0.090**	**1.62 ± 0.56**
Standard DQN	1.027 ± 0.106	1.8% ± 1.5	0.024 ± 0.022	0.496 ± 0.056	2.37 ± 0.47
Tabular Q-learning	−0.710 ± 0.595	2.8% ± 0.7	0.184 ± 0.066	0.542 ± 0.035	3.92 ± 0.15
Rule-based (max 13)	−41.574	50%	4.40	0.110	0.00

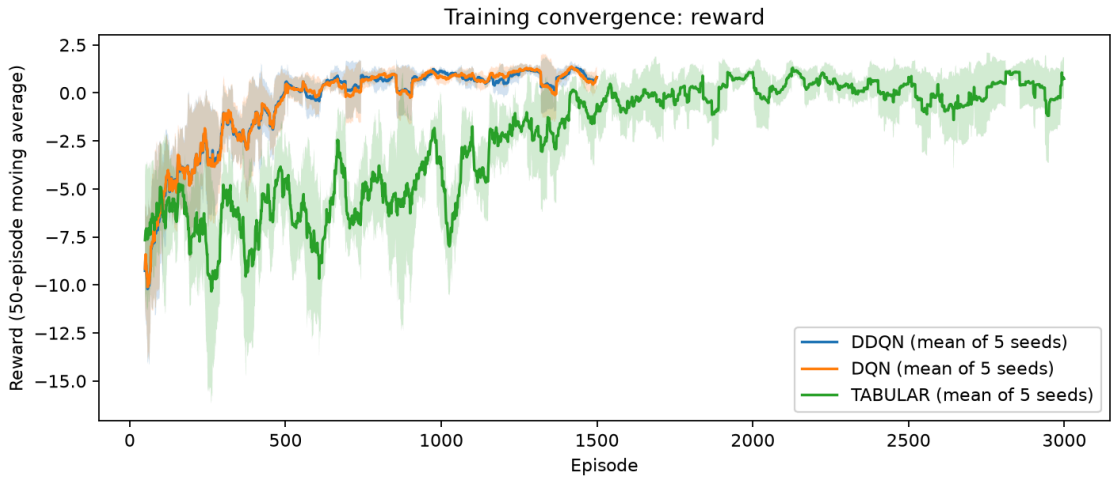
100 test sessions per seed, mean ± std over 5 seeds. Rate jump = average change of the UF rate between steps (mL/kg/h).

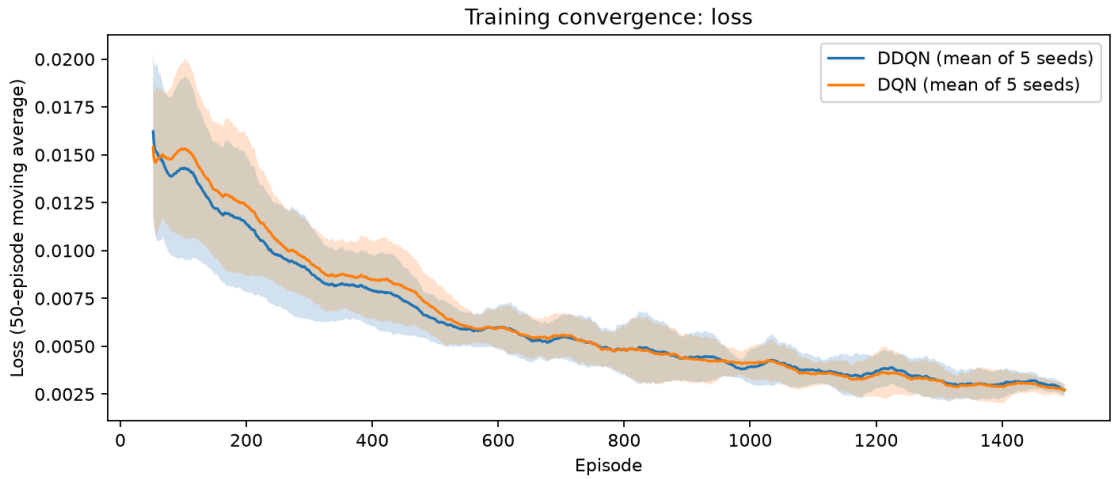
≈ DDQN and DQN: same reward

1.0% low-BP sessions for DDQN (DQN: 1.8%)

50% rule: low BP in half the sessions

Learning curves and checks (before IG)





✓ Converged

Rewards stopped changing, on all 5 seeds

✓ Stable

5 seeds agree: spread 0.25 < 0.5

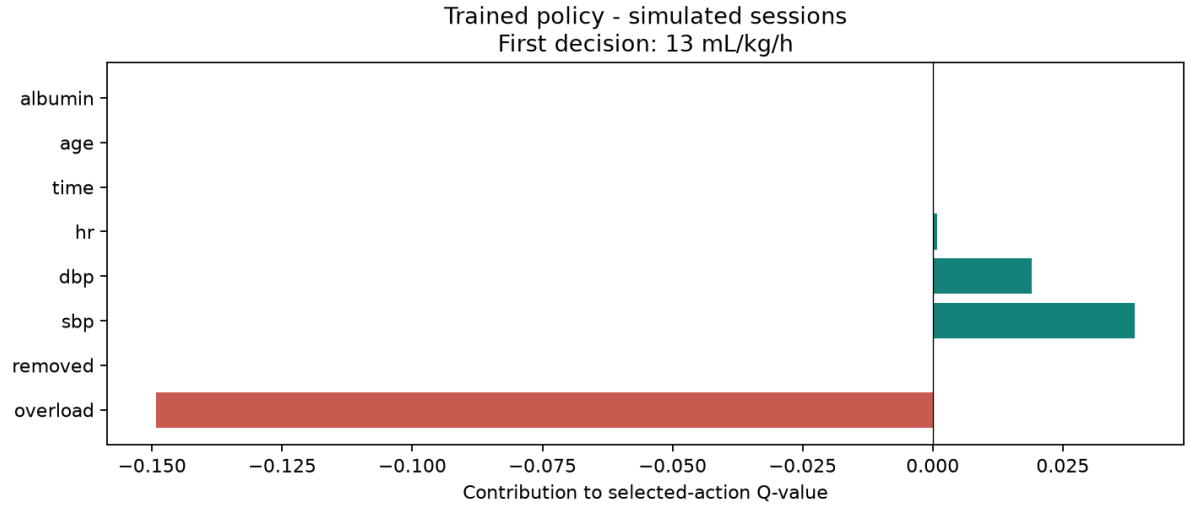
✓ Safe

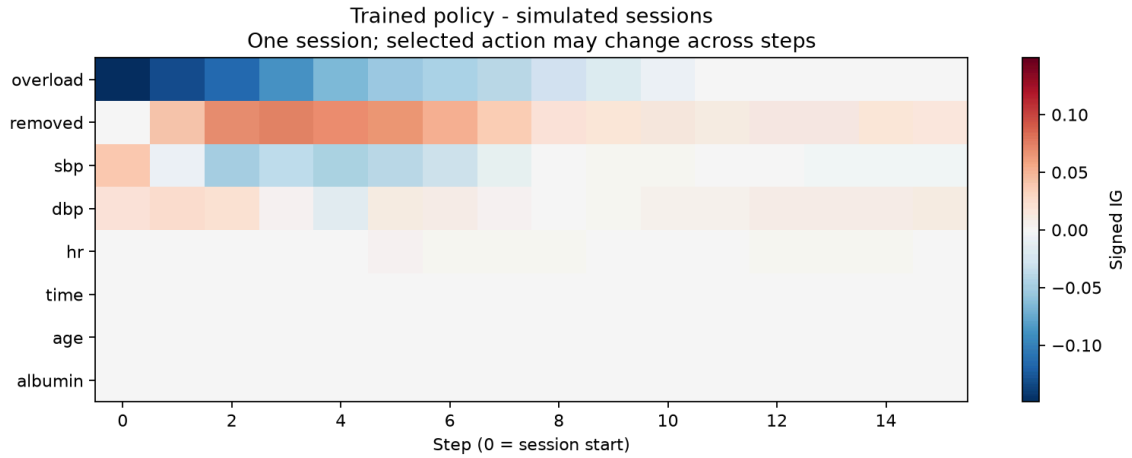
Few unsafe sessions: worst seed 4% ≤ 5%

APPROVED for IG

DDQN, 5 seeds, 100 test sessions per seed (results/validation_report.md)

Integrated Gradients, simply





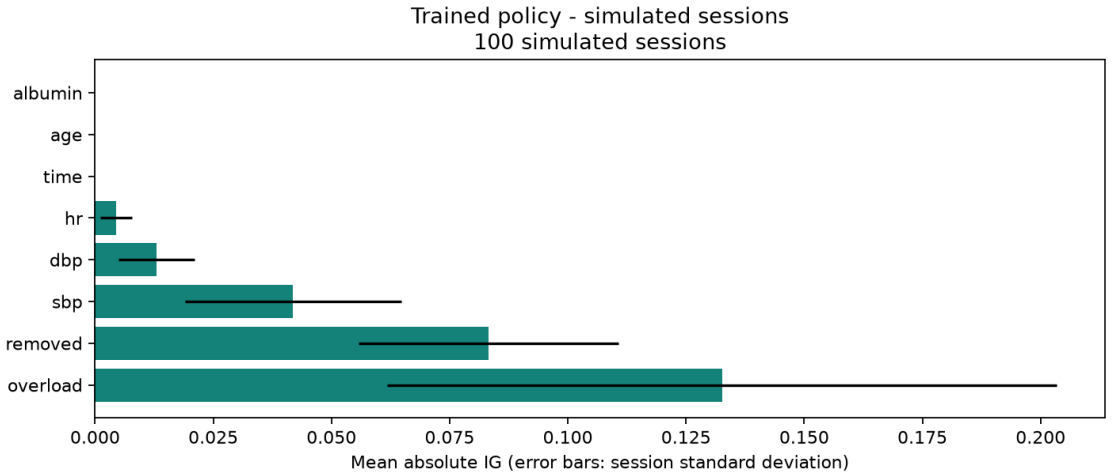
$IG_i = (x_i - b_i) \cdot \int_0^1 \partial Q_a / \partial x_i \, d\alpha$ [3]
= how much input i pushed the chosen action, compared with a reference patient b (the IG baseline).

Top: one decision (rate 13)
Water left pulls it down; blood pressure pushes it up.

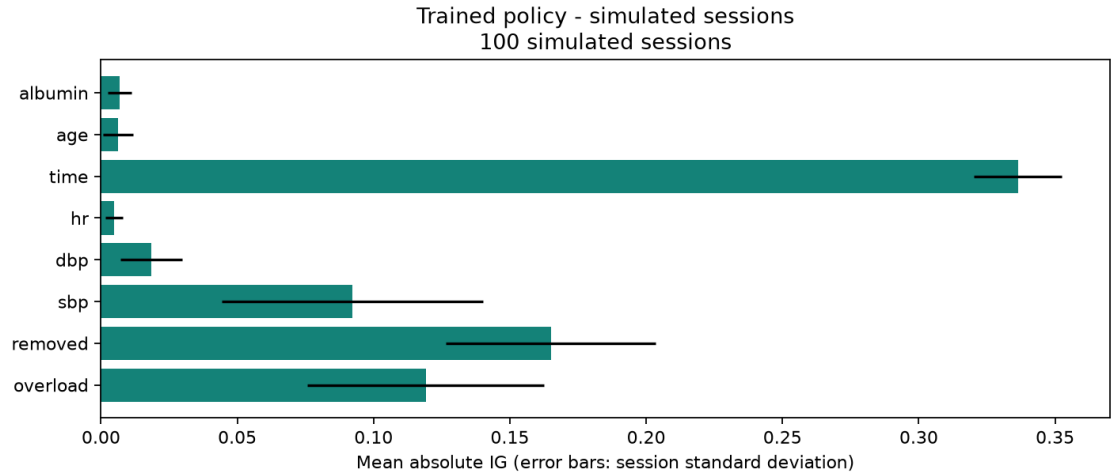
Bottom: one session
Water left matters most early, then fades.

The parts add up to the Q-value change (checked on every decision).

IG across 1600 decisions



Reference 1: default (no extra water)



Reference 2: average training patient

mean |IG|	overload	removed	sbp	dbp	hr	time	age	albumin
Default	0.133	0.083	0.042	0.013	0.004	0*	0*	0*
Average	0.119	0.165	0.092	0.018	0.005	0.336	0.006	0.007

** zero by design: this reference keeps time, age and albumin equal to the patient's own values*

The agent uses water left, water removed and blood pressure most; age and albumin barely matter.

Discussion

What we found

- DDQN ≈ DQN on reward (1.022 vs 1.027)
- DDQN is safer: 1.0% vs 1.8% low-BP sessions
- DDQN is smoother: rate jump 1.62 vs 2.37
- Rule-based: most water removed, but low BP in 50% of sessions

Limits

- Simple simulator, numbers set by hand; not real sessions
- IG sessions use all 400 profiles, not only test patients
- IG shows what the network uses, not cause and effect
- IG results depend on the reference patient

Bottom line: same reward as DQN, but safer and smoother, in simulation only.

Future work

1

Prioritized replay

Learn more often from steps with big errors.

2

Dueling network

Split Q into: how good the state is + how good each action is.

3

More seeds + tuning

Try other learning rates, γ, buffer sizes and ε schedules.

4

Offline RL on real data

Learn from recorded real sessions (Lin et al. 2019).

5

Contrastive IG

Explain why action A and not action B.

References (1/3)

[1] Z. Yang, Y. Tian, T. Zhou, Y. Zhu, P. Zhang, J. Chen, and J. Li, “Optimization of Dry Weight Assessment in Hemodialysis Patients via Reinforcement Learning,” *IEEE Journal of Biomedical and Health Informatics*, vol. 26, no. 10, pp. 4880–4891, 2022, doi: 10.1109/JBHI.2022.3192021. https://doi.org/10.1109/JBHI.2022.3192021

[2] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” in *Proceedings of AAAI*, 2016. Original paper: https://arxiv.org/abs/1509.06461

[3] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic Attribution for Deep Networks,” in *Proceedings of ICML*, PMLR, vol. 70, pp. 3319–3328, 2017. https://proceedings.mlr.press/v70/sundararajan17a.html

[4] Z. Li, S. Hao, S. Shi, L. Li, and Z. Tao, “An explainable machine learning model for early warning of hypertensive and hypotensive anomalies in maintenance hemodialysis patients,” *BMC Nephrology*, vol. 26, art. 318, 2025, doi: 10.1186/s12882-025-04270-5. https://link.springer.com/article/10.1186/s12882-025-04270-5 (Publisher notes a September 2025 correction.)

References (2/3)

[5] P. Wang et al., “Construction and application of machine learning models for predicting intradialytic hypotension,” *PLOS One*, vol. 20, no. 10, e0333357, 2025, doi: 10.1371/journal.pone.0333357. https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0333357

[6] M. Kanbay, L. A. Ertuglu, B. Afsar, E. Ozdogan, D. Siriopol, A. Covic, C. Basile, and A. Ortiz, “An update review of intradialytic hypotension: concept, risk factors, clinical implications and management,” *Clin. Kidney J.*, vol. 13, no. 6, pp. 981–993, 2020, doi: 10.1093/ckj/sfaa078. https://doi.org/10.1093/ckj/sfaa078

[7] L. Rubini, P. Soundarapandian, and P. Eswaran, “Chronic Kidney Disease,” UCI Machine Learning Repository, 2015, doi: 10.24432/C5G020. https://doi.org/10.24432/C5G020

References (3/3)

[8] H. Polat, H. Danaei Mehr, and A. Cetin, “Diagnosis of chronic kidney disease based on support vector machine by feature selection methods,” *J. Med. Syst.*, vol. 41, no. 4, art. 55, 2017, doi: 10.1007/s10916-017-0703-x. https://doi.org/10.1007/s10916-017-0703-x

[9] W. Cheungpasitporn, A. Athavale, L. Ghazi, K. B. Kashani, T. Colicchio, J. L. Koyner, J. Chen, J. H. Ix, G. Nadkarni, and J. A. Neyra, “Transforming nephrology through artificial intelligence: a state-of-the-art roadmap for clinical integration,” *Clin. Kidney J.*, vol. 19, no. 2, art. sfag004, 2026, doi: 10.1093/ckj/sfag004. https://doi.org/10.1093/ckj/sfag004

[10] Z. Tang, P. Sun, R. Liang, S. Tong, Y. Yang, M. Yang, and H. Chen, “Reinforcement learning for treatment decision-making in sepsis: a scoping review,” *npj Digit. Med.*, 2026, doi: 10.1038/s41746-026-03034-1. https://doi.org/10.1038/s41746-026-03034-1